CMake Tutorial Verification on Windows Server 2025

Analysis of CI-Driven Build System Testing

2026-03-09 (CI Run Timestamp)

Abstract

This document analyzes a GitHub Actions workflow that systematically validates the CMake build system by executing the official CMake Tutorial suite on a **Windows Server 2025** infrastructure. The workflow uses recursive `make` invocations to test specific CMake capabilities including **Generators**, **Add Executable**, **Add Library**, and **Target Link Libraries**. The analysis includes the actual source files and build configuration to demonstrate the verification process.

1 Introduction

This CI pipeline treats the CMake Tutorial not as documentation, but as a set of unit tests for the build system itself. By recursively calling `make` on isolated tutorial modules, the workflow transforms documentation into a comprehensive verification suite that confirms:

- CMake can generate Unix Makefiles on Windows
- The MinGW64 toolchain (GCC 15.2.0) functions correctly
- Static library generation and linking work properly
- Error handling (non-zero exit codes) is managed correctly

2 Environment and Tooling

The execution environment is characterized by a very recent software stack:

- **Operating System:** `Microsoft Windows Server 2025` (Build 10.0.26100)
- **Runner Image:** `windows-2025` (Version `20260302.43.1`)
- **CMake Version:** `3.31.6`
- **Compiler:** GNU GCC `15.2.0` (MinGW64)
- **Make:** GNU Make `4.4.1`
- **Git:** `2.53.0.windows.1`

```
2026-03-09T17:32:29.5260023Z Current runner version: '2.332.0'
2026-03-09T17:32:29.5284100Z ##[group]Runner Image Provisioner
2026-03-09T17:32:29.5284948Z Hosted Compute Agent
```

3 File System Hierarchy

The tutorial is organized in a hierarchical structure based on the directories referenced in the build logs:

```
tutorial/
  generators/
    UNIX/
      add_excutable/
        src/
          main.cc
      add_library/
        include/
          42.h
          answer.h
        lib/
          answer.cc
      target_link_libraries/
        include/
          42.h
          answer.h
        lib/
          answer.cc
        src/
          main.cc
```

4 Workflow Architecture: Recursive Make

The core mechanism of this CI pipeline is the recursive invocation of `make`. The top-level script runs `make -C tutorial`, which in turn invokes `make -C [submodule]`. This architecture allows the CI to isolate and verify specific CMake features without polluting the global build state.

4.1 The Execution Tree

The log reveals a four-level nesting structure:

1. **Level 1:** `make` (Top-level entry)
2. **Level 2:** `make -C tutorial`
3. **Level 3:** `make -C generators`
4. **Level 4:** `make -C [UNIX, add_excutable, add_library, target_link_libraries]`

5 Tutorial Modules - Source Code

5.1 Add Executable Module

This module demonstrates a simple executable returning an exit code.

Listing 1: src/main.cc

```
int main () { return 42; }
```

5.2 Add Library Module

This module demonstrates static library creation.

Listing 2: include/42.h

```
#ifndef ANSWER
#define ANSWER 42
#endif
```

Listing 3: include/answer.h

```
#ifndef ANSWER_H
int answer();
#endif
```

Listing 4: lib/answer.cc

```
#include "answer.h"
#include "42.h"
int answer () { return ANSWER; }
```

5.3 Target Link Libraries Module

This module demonstrates linking an executable against a static library.

Listing 5: src/main.cc (modified)

```
#include "answer.h"
int main () { return answer(); }
```

6 Build System Configuration

6.1 CMakeLists.txt Structure

While the logs primarily show the execution output, the build system is driven by CMakeLists.txt files in each directory. Based on the tutorial steps and compiler detection, the configuration corresponds to the following logic:

Listing 6: Standard CMakeLists.txt Structure

```
cmake_minimum_required(VERSION 3.31)
project(tutorial)

# Add Executable
add_executable(main src/main.cc)

# Add Library
add_library(answer lib/answer.cc)

# Target Link Libraries
target_link_libraries(main answer)
```

6.2 Generated Makefile

The generated Makefile confirms proper CMake configuration and path handling.

Listing 7: Generated Makefile Structure

```
default_target: all
.PHONY : default_target

CMAKE_COMMAND = "C:/Program Files/CMake/bin/cmake.exe"
CMAKE_SOURCE_DIR = D:/a/tutorial/generators/UNIX
CMAKE_BINARY_DIR = D:/a/tutorial/generators/UNIX/build
SHELL = /bin/sh
```

7 Build System Analysis

7.1 Unix Makefiles on Windows

A critical aspect of this workflow is the explicit selection of the **Unix Makefiles** generator on a **Windows** OS:

```
cmake -G "Unix Makefiles" -B build
...
SHELL = /bin/sh
```

This verifies that the MinGW environment correctly supports Unix-style scripts and path handling (forward slashes) within a native Windows context.

```
find build | cat -n
      1 build
      2 build/CMakeCache.txt
      3 build/CMakeFiles
      4 build/CMakeFiles/3.31.6
      5 build/CMakeFiles/3.31.6/CMakeCCompiler.cmake
      6 build/CMakeFiles/3.31.6/CMakeCXXCompiler.cmake
      7 build/CMakeFiles/3.31.6/CMakeDetermineCompilerABI_C.bin
      8 build/CMakeFiles/3.31.6/CMakeDetermineCompilerABI_CXX.bin
      9 build/CMakeFiles/3.31.6/CMakeRCCompiler.cmake
     10 build/CMakeFiles/3.31.6/CMakeSystem.cmake
```

8 Observations

8.1 Exit Code 42

The workflow explicitly handles non-zero exit codes. In the `add_excutable` module, the program `main` is configured to return 42:

```
# The will return
# make: [all] Error 42 (ignored)
./build/main
```

This demonstrates that the CI pipeline is robust enough to treat this specific exit code as a successful validation of error-handling logic rather than a build failure.

8.2 CMake Configuration Log

The CMake configuration output shows successful compiler detection:

```
-- The C compiler identification is GNU 15.2.0
-- The CXX compiler identification is GNU 15.2.0
-- Detecting C compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Check for working C compiler: C:/mingw64/bin/cc.exe - skipped
-- Check for working CXX compiler: C:/mingw64/bin/c++.exe - skipped
-- Configuring done (8.7s)
-- Generating done (0.0s)
-- Build files have been written to: ...
```

9 Conclusion

The workflow validates that CMake can function correctly on Windows Server 2025 with the following key takeaways:

1. **Portability:** Unix Makefiles function correctly on Windows Server 2025 with MinGW.
2. **Modernity:** The environment supports the latest versions of GCC (15.x) and CMake (3.31.x).
3. **Resilience:** The pipeline successfully manages non-zero exit codes associated with tutorial logic.
4. **Verification Suite:** The CMake Tutorial is transformed from documentation into a comprehensive verification suite.

The actual source code demonstrates the hierarchical structure of CMake projects and confirms that the build system correctly handles:

- Executable compilation
- Static library generation
- Linking executables against libraries
- Cross-platform makefile generation